

OPENLEARN V2 · 测试协作手册

平台部署与插件 开发测试上手指南

本手册面向参与平台与插件测试的朋友，覆盖从环境准备、创建插件、打包安装、迭代修改到版本更新的完整流程。按章节顺序操作，每一步都给出了可直接执行的命令。

适用版本：openlearn-next v0.3.x (宿主 API 0.2.9)

预计上手时间：约 1 小时

目录 CONTENTS

- | | |
|--------------|--------------|
| 1. 环境准备与平台启动 | 5. 迭代修改与本地测试 |
| 2. 认识插件体系 | 6. 插件更新与卸载 |
| 3. 创建你的第一个插件 | 7. 常见问题与检查清单 |
| 4. 打包与安装插件 | |

一、环境准备与平台启动

1.1 前置要求

- **Node.js ≥ 20**（自带 npm 与 npx，推荐方式无需 clone 源码即可启动平台）；操作系统不限（Linux / macOS / Windows + WSL）
- 如需插件**热重载迭代**或平台源码调试，再选源码方式（另需 **pnpm ≥ 9** 与 Git）

1.2 启动平台（首选：npx 一键运行）

无需克隆项目，首次运行自动下载平台包；npx 方式数据库默认在 `~/openlearn-next/data.db`，与源码目录完全隔离：

```
npx openlearn-next          # 启动平台，默认端口 9000
npx openlearn-next -p 3000  # 自定义端口
OPENLEARN_DB_PATH=./my.db npx openlearn-next  # 自定义数据库路径
```

```
# 备选源码方式（仅插件热重载迭代 / 平台源码调试时需要）：
git clone https://github.com/aymwoo/OpenLearn-Next-V2.git
cd OpenLearn-Next-V2 && pnpm install
pnpm dev      # 开发服务器，数据库在 packages/core/db/educational_os.db
```

启动成功的标志：终端输出 `listening on port 9000`，浏览器访问 `http://localhost:9000`。

1.3 默认账号

账号	密码	权限说明
admin	admin	管理员，拥有插件中心（Plugin Center）等全部功能
teacher	teacher	教师账号，可体验课程、白板与插件 UI 扩展点

1.4 常用命令

命令	用途
<code>npx openlearn-next</code>	npx 方式启动平台（测试朋友的首选）
<code>npm update -g openlearn-next</code>	全局安装方式下更新平台（npx 方式重新运行即自动拉取最新版）
<code>pnpm test</code>	运行 Vitest 测试套件（源码方式）
<code>pnpm db:reset</code>	重置数据库（源码方式）

小提示：数据搞乱了？npx 方式删除 `~/openlearn-next/data.db` 后重启即可；源码方式用 `pnpm db:reset`。

二、认识插件体系

每个插件都是一个独立包，核心是「manifest 清单 + activate 激活函数」。平台通过 ESM Loader 加载插件，按 7 状态生命周期管理：`installed` → `activating` → `active` ⇌ `deactivating` → `inactive` → ... → `uninstalled`，激活或停用失败会自动回滚并标记为 `error`。

2.1 manifest.json 必需字段

```
{
  "id": "ext-homework-hub",           // 必需: 全局唯一 ID (建议 ext- 前缀或 @org/name)
  "name": "作业管理中心",           // 必需: 显示名称
  "version": "1.0.0",               // 必需: SemVer 版本号, 更新插件靠它
  "main": "dist/index.js",          // 必需: 入口文件相对路径
  "engines": { "openlearn": ">=0.2.5" } // 宿主 API 版本兼容范围
}
```

2.2 常用可选字段

字段	作用
<code>requires / optional</code>	强/弱依赖的平台服务，如 <code>@openlearn/core:ICommandBusService@^1.0.0</code> ； <code>optional</code> 缺失时服务为 <code>null</code> 而不报错
<code>pluginDependencies</code>	依赖的其他插件，须先处于 ACTIVE 状态
<code>capabilitiesProposed</code>	申请的能力权限，如 <code>storage:read</code> 、 <code>ai:chat</code> 、 <code>vfs:write</code>
<code>contributes</code>	UI 贡献点声明（ <code>teacher.tab</code> 、 <code>classroom.tool</code> 、 <code>student.view</code> 等），实际渲染需在 <code>activate</code> 中用 <code>ctx.ui.registerExtensionPoint()</code> 注册
<code>configuration</code>	插件配置项 schema，管理员可在插件中心配置

2.3 插件运行时上下文 (ctx)

`activate` 函数收到的 `ctx` 提供：`ctx.services`（命令总线、事件总线、AI、存储等）、`ctx.db`（插件私有 SQLite 表）、`ctx.ui`（UI 扩展点注册）、`ctx.logger`、`ctx.require()`（加载平台内置的第三方库，如 `uuid`、`xlsx`、`recharts`）。

深入阅读：manifest 完整规范见 `docs/plugin/plugin-manifest-spec.md`；生命周期细节见 `docs/plugin/plugin-lifecycle.md`；完整教程见 `docs/tutorials/plugin-development-tutorial.md`。

三、创建你的第一个插件（首选：AI Agent 开发）

首选方式：向 AI Agent 提出需求即可。AI Agent（如 ZCode）安装 `openlearn-next-plugin-dev skill`（skillhub.cn/skills/user_37fa416d/openlearn-next-plugin-dev）后，会自动探活并获取最新插件开发在线文档（带本地缓存），与开发者讨论需求后直接生成代码、构建打包。

1 在 AI Agent 会话中直接描述需求：

```
使用 openlearn-next-plugin-dev skill,  
开发一个课堂签到插件：含签到按钮和实时统计
```

2 Agent 自动完成整个开发流程：

- 加载 skill，探活在线文档（`openlearn-next-v2.readthedocs.io`），缓存命中则零额外 token 消耗；
- 读取插件开发指南（Manifest 规范、命令-事件-Action 模式、UI 扩展槽位、安全权限模型）；
- 生成 `src/index.ts`（manifest + activate，注册命令）与 `src/frontend.tsx`（前端面板）；
- 执行 `npm @openlearn/plugin-sdk build` 构建并打包出 `.zip` 安装包。

3 备选：手动脚手架（想了解内部原理时使用）——入口 `src/index.ts` 默认导出 `{ manifest, activate }`，manifest 必填 `id / name / version / main` 四个字段：

```
npm @openlearn/plugin-sdk init --name my-plugin --template full-stack  
cd my-plugin && npm install  
npm @openlearn/plugin-sdk build # esbuild 打包并生成 my-plugin.zip
```

三种模板：`server-only`（默认）、`full-stack`（后端 + 前端 UI）、`frontend-only`。

注意（token-enforcer 安全扫描）：插件入口只允许「相对路径导入」和 `@openlearn/*` Token 导入，`crypto`、`fs`、`path` 等 Node 模块和 npm 包裸导入会被拒绝。已知问题：当前脚手架 CLI 会把 SDK 误打进产物，若遇「Import of "node:fs" is not allowed」，让 AI Agent 以 `external: ['@openlearn/plugin-sdk']` 重新打包即可（与平台官方构建方式一致）。

四、打包与安装插件

4.1 通过插件中心安装（推荐）

- 1 用 admin / admin 登录平台，进入 系统设置 → 插件中心（Plugin Center）。
- 2 点击上传，选择 my-plugin.zip ；或使用「一键极速安装（Express Install）」向导。
- 3 系统会自动完成：解压 → 解析 manifest → 安全扫描（token-enforcer）→ SemVer 兼容检查 → 权限风险提示（红 = 高危 / 琥珀 = 中 / 绿 = 安全）→ 入库 → 可选立即激活。
- 4 在插件列表中确认状态为 active。若激活失败，列表中会显示 error 状态与错误信息，这是测试时需要重点报告的内容。

4.2 通过 REST API 安装（调试用）

```
curl -F "file=@my-plugin.zip" http://localhost:9000/api/plugins/upload
curl http://localhost:9000/api/plugins # 查看已安装插件列表
curl -X POST http://localhost:9000/api/plugins/ext-my-plugin/toggle # 激活/停用切换
```

接口	说明
GET /api/plugins	已安装插件列表
POST /api/plugins/upload	上传 ZIP 安装（同 ID 不同版本走更新路径）
POST /api/plugins/:id/toggle	激活 / 停用切换
DELETE /api/plugins/:id	卸载插件
GET/POST /api/plugins/:id/config	读取 / 写入插件配置
GET /api/plugins/store	插件市场可安装列表

安装后去了哪里： 插件文件解压在 plugins/<uuid>/ 目录，元数据记录在 SQLite 的 plugins 表（状态：installed / active / inactive / error）。

五、迭代修改与本地测试

5.1 首选：对话式迭代（AI Agent）

修改插件**不需要自己改代码**——继续在 AI Agent 会话里提需求即可，Agent 会按 skill 指引自动完成「改代码 → 升版本号 → 重新打包」：

再加一个签到趋势图表，然后升版本重新打包

Agent 的典型动作：`Edit src/frontend.tsx`（加图表）→ `manifest.json version: 0.1.0 → 0.1.1` → `npx @openlearn/plugin-sdk build` → 产出新 ZIP。之后把新 ZIP 重新上传插件中心（见第六章平滑更新）。也可以直接让 Agent 连浏览器操作，完成上传与验证。

5.2 热重载（源码方式）

在源码方式（`pnpm dev`，即 `NODE_ENV=development`）下，PluginHost 会监听 `plugins/` 目录，文件变更约 300ms 后自动执行「停用旧版 → 清理 → 激活新版」，**无需重启服务器**。

注意：热重载覆盖后端逻辑；前端 UI 组件的改动需重新构建入口文件并刷新页面。

5.3 单元测试（Vitest + Mock Context）

```
npm install --save-dev @openlearn/plugin-test-kit vitest
npx vitest run                # 在插件项目内运行
pnpm test                     # 在平台仓库根运行全量测试
```

```
// __tests__/index.test.ts
import { describe, it, expect } from 'vitest';
import { createMockContext } from '@openlearn/plugin-test-kit';
import plugin from '../src/index';

describe('my-plugin', () => {
  it('activates and registers handler', async () => {
    const ctx = createMockContext({
      pluginId: 'ext-my-plugin',
      capabilities: ['lesson:read'],
      overrides: { ai: { generateText: async () => 'mocked' } },
    });
    await plugin.activate(ctx);
    expect(ctx.services.commandBus.handlers.get('myplugin.hello')).toBeDefined();
  });
});
```

`createMockContext()` 提供命令总线、事件总线、AI、存储、日志等全套 Mock，其中 `ctx.db` 是真实内存 SQLite，`ctx.services.commandBus.executeCalls` 可作为断言依据。测试同样可以交给 AI Agent：「为这个插件补一个 `createMockContext` 单元测试并跑到通过」。

六、插件更新与卸载

6.1 版本更新的正确姿势

- 1 修改代码后，务必递增 `manifest.json` 中的 `version`（如 1.0.0 → 1.1.0）。
- 2 重新执行 `npx @openlearn/plugin-sdk build` 生成新的 ZIP。
- 3 在插件中心重新上传 **相同 ID、不同版本** 的 ZIP，系统自动走更新路径：
 - 若插件正处于 **ACTIVE** 状态，会自动「停用 → 覆盖文件与数据库记录 → 重新激活」，平滑无感升级；
 - 若处于其他状态，则只覆盖文件，等待下次激活。

6.2 停用与卸载

- **停用**：插件中心列表中切换开关，或调用 `POST /api/plugins/:id/toggle`；
- **卸载**：插件中心卸载按钮，或 `DELETE /api/plugins/:id`，文件与数据库记录一并删除；
- 也可以「卸载后重新安装」当作最粗暴的更新方式，但会丢失插件私有数据。

6.3 测试时建议覆盖的场景

场景	预期表现
首次安装并激活	状态变为 active，扩展点正常渲染
激活后刷新页面	UI 扩展点仍然存在，配置不丢失
停用后再激活	功能恢复，无残留副作用
上传同 ID 更高版本	自动平滑升级，配置保留
上传含非法导入的 ZIP	安全扫描拦截，给出明确报错信息
卸载后重新安装	回到初始状态，不报错

七、常见问题与检查清单

7.1 常见问题 (FAQ)

现象	排查方法
上传 ZIP 后激活失败	看插件中心报错与服务器终端日志；检查 <code>engines.openlearn</code> 版本范围是否包含 0.2.9
安全扫描拒绝安装	入口存在裸导入（ <code>fs/crypto/npm</code> 包），改用 <code>ctx.require()</code> 或全局 API，重新打包
改了代码没生效	后端逻辑需重新 build + 上传（或放 <code>plugins/</code> 目录走热重载）；前端组件需刷新页面
插件列表找不到插件	<code>curl http://localhost:9000/api/plugins</code> 确认安装状态；检查数据库是否被 <code>pnpm db:reset</code> 清空
依赖的服务是 null	检查 manifest 的 <code>requires / optional</code> 服务 Token 拼写与版本范围

7.2 上手检查清单

- ☐ 平台启动成功，能访问 `http://localhost:9000` 并用 `admin/admin` 登录
- ☐ 用脚手架创建了插件（`init` → `npm install`）
- ☐ manifest 包含 4 个必需字段且 ID 唯一
- ☐ `plugin-sdk build` 生成 ZIP 成功
- ☐ 通过插件中心上传并激活成功（状态 `active`）
- ☐ 修改代码、升版本号、重打包、重上传，完成一次完整迭代
- ☐ 至少跑通一个 `createMockContext` 单元测试
- ☐ 测试过停用 → 激活 → 卸载 → 重装闭环

7.3 问题反馈格式建议

提交问题时请附上：插件 ID 与版本号、复现步骤、预期行为、实际行为、服务器终端日志截图（含 `[PluginHost]` 相关行）。

更多资料：完整插件开发教程 `docs/tutorials/plugin-development-tutorial.md` · Manifest 规范 `docs/plugin/plugin-manifest-spec.md` · SDK 说明 `docs/sdk/plugin-sdk.md` · 平台快速开始 `docs/getting-started/quickstart.md`

祝测试顺利！遇到文档未覆盖的情况，欢迎直接联系平台维护者反馈。